

EXAMEN DE FIN D'ÉTUDES SECONDAIRES – Sessions 2024

QUESTIONNAIRE

Date :	07.06.24	Horaire :	08:15 - 11:15	Durée :	180 minutes
Discipline :	SCIPR	Type :	écrit	Section(s) :	CI
					Numéro du candidat :

Modelling part

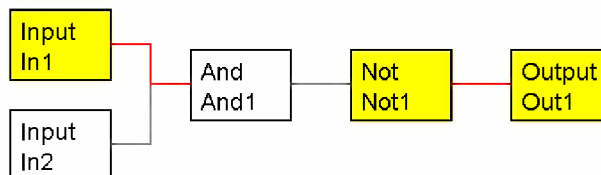
10 points

- The modelling part must be written on paper and must be handed in before the implementation part can start.
- The modelling part is due after 35 minutes at the latest, but you are free to hand it in earlier.

You are required to setup the model of a program that can be used to simulate a digital circuit with elements from the following list:

- An input that the user can activate or deactivate with a click of the mouse.
- An output.
- A NOT gate or inverter whose output is always the inverse of its input.
- An AND gate whose output is always the logical sum of all its inputs taken with the "AND" operator (the output is **active** if all the inputs are active, inactive otherwise).
- An OR gate whose output is always the logical sum of all its inputs taken with the "OR" operator (the output is **inactive** if all the inputs are inactive, active otherwise).

Here is an example:



Explications:

- Each element has a unique name and a state ("active" or "non-active").
- In1 and In2 are inputs and In1 is active.
- And1 is an AND gate, and as not all the inputs are active, so its output is inactive.
- Not1 is an inverter. As its input is inactive, its output is active.
- Out1 is an output and is active because its input is active.

The following information is also available:

- Each element has a limited number of inputs and an unlimited number of outputs.
- The output of an element is evaluated according to its type.
- If an element changes state, all connected elements need to be reevaluated.
- The size of the various elements is hard coded in the program.

Use the principles of object-oriented programming to create class diagram of the application model on paper, using the usual UML conventions.

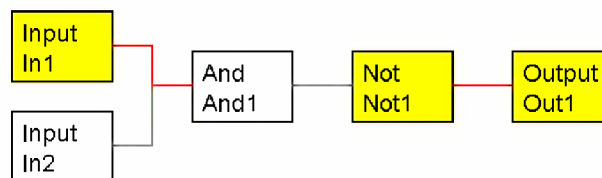
Adding comments onto your diagram is recommended in order to explain your choice!

- Add the necessary classes for the different types of elements!
- First, limit yourself to the basic attributes of these classes!
- Add the attributes needed to link these elements together!
- Add a class (with attributes and methods) representing the digital circuit and choose a suitable data structure for storing the elements. Note that this class must perform the following operations as efficiently as possible:
 - a. draw all the elements,
 - b. test whether the circuit already has an element with a given name,
 - c. search for an element with a coordinate in its drawing area,
 - d. save all the elements, line by line, in a text file,
 - e. load the elements from a text file.

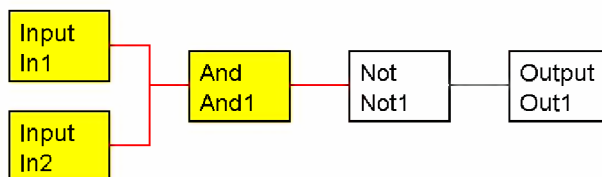
Add a comment to explain your choice of the data structure for storing elements!

- If an input changes state, it must be possible to transmit this modification to all connected elements. Add the necessary method(s)!

Consider the following example:



By activating In2, the connected elements change state:



In your working directory (to be defined by each school), you will find a folder called EXAMEN_CI. Rename this folder by replacing the current name with your exam code (example of notation: LXY_CI2_05). All your files should be saved in this folder, which will be called “your folder” afterwards.

Implementation part

50 points

Next, you will code a program to simulate simple digital circuits. The skeleton of this program is already in your folder. Open it and complete it according to the following instructions.

To simplify things, we will assume that the initial states of the elements in the `codeTextArea` component, and therefore also those in the save files, are always correct.

All exceptions must be handled in the main `MainFrame`, i.e. all possible exceptions are passed on to the calling class.

The “Gate” class

10.5 points

This abstract class represents a generic element of a digital circuit. Each element has a limited number of inputs and an undefined number of outputs. It has the following attributes, which are already present in the code:

0 points

- `HEIGHT` its height, the value of which is fixed at 35
- `WIDTH` its width, set to 50
- `alOutputs` the list of all elements to which this element is connected
- `alInputs` the list of all elements whose output is connected to this element
- `maxInputCount` the maximum number of inputs this element can have
- `id` its identifier
- `x, y` its position (top left)
- `state` its state (true = active, false = non-active)

Besides the constructor, which initialises the some attributes, this abstract class has the following methods:

1 point

- The abstract `evaluate()` method is used to evaluate the state of an element. 0.5 point
- The `connectOutputTo(...)` method connects the output of this element to another element if, and only if, the maximum number of inputs to the other element has not been reached. Otherwise, an exception of type `MaximumInputLimitReachedException` with an understandable explanatory message must be thrown. To connect one element to another, the current element must be added to the list of entries for the element passed as a parameter. 2.5 points
- The class implements the `Comparable<Gate>` interface. It therefore has a `compareTo(...)` method which compares two elements on the basis of their states according to the truth table shown below: 2 points

this.state other.state	true	false
true	0	-1
false	1	0

- The `getCsv()` method returns a purely textual representation of the element in CSV format in the format shown below: **2.5 points**

`<class name>,<ID>,<x>,<y>,<état (1 if active, 0 if not active)>,<ID of the output #1>,<ID of the output #2>,...`

Example: `Input,In1,10,10,0,And1,Or7`

Explanation: This is an element of the `Input` class whose identifier is "In1". The element is located at position (10,10) and is inactive. Its output is connected to two elements whose names are "And1" and "Or7".

Note: The code `getClass().getSimpleName()` is used to extract the name of the class from which the current object has been instantiated.

- The `isInside(...)` method tests whether the point passed as a parameter is inside the rectangle (`x,y,WIDTH,HEIGHT`) defined by this element. **2 points**
- The `draw(...)` and `drawConnectionTo(...)` methods, which draw the element and the links between the different elements, are given. **0 points**

The “Input” class

1.5 points

This class inherits from the `Gate` class and represents an input element.

0.5 points

- The `evaluate()` method evaluates the state of all the elements connected to this element. **1 point**

The “Output” class

3 points

This class inherits from the `Gate` class and represents an output element with a single input.

0.5 points

- If the element has an input, the `evaluate()` method updates the state of the element (new state → state of its input). If the state has changed, all the elements connected to this element are evaluated. **2.5 points**

The “Not” class

1 points

This class inherits from the `Gate` class and represents an element which inverts the state of its single input.

- If the element has an input, the `evaluate()` method updates the state of the element (new state → inverted state of its input). If the state has changed, all elements connected to this element are evaluated.

1 point

The “And” class**2.5 points**

This class inherits from the `Gate` class and represents an element that combines all its inputs using a logical "and" operator.

- This element has a maximum of 3 inputs. **0.5 point**
- If the element has at least one input, the `evaluate()` method updates the state of the element (new state → states of all its inputs combined using an "and" operator). If the state has changed, all connected elements are evaluated. **2 points**

Remember: The output is active if and only if all the inputs are active.

The “Or” class**1.5 points**

This class inherits from the `Gate` class and represents an element that combines all its inputs using an "or" logical operator.

- This element has a maximum of 2 inputs. **0.5 point**
- If the element has at least one input, the `evaluate()` method updates the state of the element (new state → states of all its inputs combined using an "or" operator). If the state has changed, all connected elements are evaluated. **1 point**

Remember: The output is active if and only if at least one input is active.

The “Diagram” class**21 points**

This class represents a digital circuit. It has the following attribute:

- `allElements` a list of all the elements of the circuit

0.5 point

The class has the following methods:

- The `getElementById(...)` method returns the element whose identifier is the one passed as a parameter, `null` otherwise. **1.5 points**
- The `isValid(...)` method tests whether the identifier passed as a parameter is valid. An identifier is valid if there isn't already another element with the same identifier and if it doesn't contain a "," (comma) symbol. **1.5 point**
- The `add(...)` method adds an element to the list, but only if its identifier is valid. In any other case, an exception of the type `InvalidIdException` with an appropriate message is thrown. **1.5 point**
- The `draw()` method draws all the elements in the list. **0.5 point**
- The `toggleInput(...)` method searches which of the `Input` elements is located in a way that the position passed as parameter belongs to its inside and, if found, inverts its state. **2.5 points**
- The `getCsv()` method returns a text version of the digital circuit. Each element of the digital circuit represents a single line in the file. **1 point**

Reminder: A carriage return (FR: retour à la ligne) is represented by the character string: `"\\n"`

Example: `"abc\\nxyz"` will be shown as

1	abc
2	xyz

in the editor.

- The `saveToFile(...)` method saves the current circuit in CSV format in the file passed as a parameter. **1 point**
- The `loadFromString(...)` method loads a digital circuit from a CSV text. The format is that described in the `Gate.toCsv()` method. **9.5 points**

In the first step, all the items are created and added to the list.

(4.5 points)

In a second step, the links between the elements are configured. During this stage, it is possible that a reference to an element identifier is not found. In this case, an exception of type `InvalidIdException` with an appropriate message should be thrown. **(5 points)**

- The `loadFromFile(...)` method loads a digital circuit from the text file passed as a parameter and interprets its contents using the `loadFromString(...)` method. **1.5 points**

The “DrawPanel” class

2 points

This class is responsible for displaying the digital circuit. It has the following attributes:

0.5 points

- `diagram` a digital circuit

Besides the setter for this attribute, you need to modify the class code so that, if possible, it draws the digital circuit on a white background. **1.5 points**

The “MainFrame” class

7 points

The graphical interface is provided and already contains source code that you need to complete following the instructions below:

- The class has an attribute representing a digital circuit. Don't forget to initialise it! **1 point**
- Clicking on the digital circuit on an element of type `Input` changes its state. **1 point**
- Clicking on the `loadButton` loads a digital circuit in text form from a CSV file into the `codeTextArea` component. The code for displaying a selection dialogue is already present. Complete the method!
If an error occurs or if the file contains an error (e.g. an identifier that is not found), an error message is displayed on the screen. Use the code snippet at the beginning of the class for this purpose. **2.5 points**
Clicking on the `saveButton` saves the current digital circuit (i.e. the one drawn in the `drawPanel` component) in text form in a file. The code for displaying a selection dialog is already present. Complete the method!
If an error occurs or if the file contains an error (e.g. an identifier that is not found), an error message will be displayed on the screen. **1 point**
- Clicking on the `drawButton` loads the text from the `codeTextArea` component into the current circuit and draws it in the `drawPanel` component.
If an error occurs or if the file contains an error (e.g. an identifier that is not found), an error message will be displayed on the screen. **1.5 point**

UML of the model

In order not to overload this diagram, some relationships are not shown.

